

BASES DE DATOS NOSQL

Bases de datos NoSQL

- Actualmente es imprescindible el manejo de una inmensa cantidad de datos que deben estar disponibles para la toma de decisiones .
- Existen alternativas a los sistemas tradicionales de gestión de bases de datos de tipo relacional, como pueden ser las bases de datos orientadas a objetos, bases de datos basadas en documentos, de pares clave-valor o grafos.
- Las bases de datos NoSQL son todas aquellas bases de datos que no siguen el modelo relacional.
- Surgen principalmente por la necesidad de contar con sistemas de almacenamiento más flexibles y dinámicos, que respondan mejor a las aplicaciones web (RRSS y contenido multimedia) y que permita obtener información de múltiples fuentes de forma más sencilla.

Bases de datos NoSQL

- Se le llama también ***Not Only SQL***, porque aunque algunos utilizan el lenguaje SQL no siguen las restricciones del modelo relacional.



Bases de datos NoSQL

- Las RDBMS son eficientes, los datos están bien estructurados y pueden ser indexados. Y son una buena alternativa si:
 - a) Las lecturas de datos son frecuentes y siguen el mismo patrón de acción
 - b) Gestionan muchas consultas pero pocas escrituras en la BD
- Algunas **limitaciones** de las RDBMS son:
 - ❏ **Coste de diseño y almacenamiento**: personal experto para el diseño, control de la coherencia frente a cambios.
 - ❏ **Poca flexibilidad y compatibilidad**: si se trata de unir datos de diferentes orígenes, o diferentes formatos o estructuras, las BD relaciones son poco flexibles.

Bases de datos NoSQL

- ❑ Poca escalabilidad para grandes volúmenes de datos: al ser tan estructurada, si crece el volumen de datos pierde eficiencia. No es apropiada para Big Data por ej.
- ❑ Limitación en la combinación de datos: si las características, tipo de datos, etc.
- ❑ Falta de interoperabilidad: poco flexibles para compartir datos entre diferentes BD
- ❑ Limitaciones con información multimedia.

Las BD NoSQL no requieren esquemas, escalan horizontalmente, gestiona mayor volumen de datos, son más flexibles con los tipos de datos.

Tipos de bases de datos NoSQL

- Según la forma de almacenar los datos, los tipos son:

Clave-valor

- El más sencillo y de rápida recuperación de los datos, cada elemento se identifica con una clave única. Los datos se almacenan como **objetos binarios** BLOB
- Ejemplos: Cassandra (Apache), Dynamo (Amazon), Redis (*Instagram*)

Documentales

- La información se almacena en forma de **documentos** (XML, JSON,...)
- Utiliza una clave única para cada registro y permite búsquedas por clave-valor
- Ejemplos: MongoDB, FireBase

En grafo

- La información se representa como nodos en un grafo y las relaciones son las aristas. Se sigue la teoría de grafos para recuperar la información de forma óptima
- Ejemplos: Neo4j, Amazon Neptune

Bases de Datos XML

Algunos SGBD incorporan mecanismos para crear y almacenar datos en formato XML, tanto relacionales como orientadas a objetos (Oracle, MySQL...)

Las Bases de datos nativas XML no almacenan datos, sino documentos XML.

Definimos una BD XML nativa (**NXD** *Native XML Database*) como un sistema de gestión de información que cumple lo siguiente:

- Define un modelo lógico para un documento XML
- Incorpora las características *ACID* (Atomicidad, consistencia, aislamiento y durabilidad)
- Incluye un número arbitrario de niveles de datos y complejidad
- Permite tecnologías de consulta y transformación del XML => XQuery, XPath...

Bases de Datos XML

XPath

Es un lenguaje que permite seleccionar nodos de un documento XML y calcular valores a partir de su contenido. Por ejemplo, las siguientes devolverían los empleados del departamento 10=>

//Empleados/empleado[deptno=10]

(existe una notación larga con :: pero en esta unidad utilizaremos la notación abreviada)

Bases de Datos XML

XQuery.

XQuery, también conocido como ***XML Query***, es un lenguaje creado para buscar y extraer elementos y atributos de documentos XML. XQuery es para XML lo que SQL es para las bases de datos. XQuery es un lenguaje de consulta diseñado para escribir consultas sobre colecciones de datos expresadas en XML. Abarca desde archivos XML hasta bases de datos relacionales con funciones de conversión de registros a XML.

Su principal función es extraer información de un conjunto de datos organizados como un árbol n-ario de etiquetas XML. En este sentido XQuery es independiente del origen de los datos.

El resultado también será un documentos XML.

XQuery hace uso de ***XPath***, un lenguaje utilizado para seleccionar partes de XML.

Bases de Datos XML

Xquery nos va a permitir:

- ▶ Seleccionar información basada en algún criterio
- ▶ Buscar información en un documento
- ▶ Unir datos desde diversos documentos o colecciones
- ▶ Organizar, agrupar o resumir datos
- ▶ Transformar o reestructurar datos XML
- ▶ ...

En XQuery las consultas siguen la norma **FLOWR** (*For, Let, Where, Order y Return*) y la sintaxis general es:

for <variable> in <expresión XPath>

let <variables vinculadas>

where <condición XPath>

order by <expresión>

return <expresión de salida>

Bases de Datos XML

Algunos ejemplos de consultas sencillas (tipo SELECT):

- Visualizar los apellidos de los empleados:
 - for \$emp in //Empleados/empleado
 - return \$emp
- Visualizar nombre + cargo, se usa una variable vinculada con los datos concatenados (en let), y crea una nueva etiqueta.
 - for \$emp in //Empleados/empleado
 - let \$vnom:=\$emp/nombre, \$vcar:=\$emp/cargo
 - order by \$emp/nombre
 - return <nombre_cargo>{concat(\$vnom,' ', \$vcar)</nombre_cargo>
- Visualizar el nombre y salario del empleado con id=3:
 - for \$emp in //Empleados/empleado[id=3]
 - return concat(\$emp/nombre,' ', \$emp/salario)

Bases de Datos XML

Funciones y operadores:

- **Matemáticos:** +, -, *, div, idiv, mod
- **Comparación:** o, !=, <, >, <=, >=, not()
- **Agrupación:** cont(), min(), max(), avg(), sum()
- **Cadena:** concat(), string-length(), starts-with(), ends-with(), substring(), string(), upper-case(), lower-case()
- **Otros:** distinct-values(), empty()...
- **Comentarios:** (:esto es un comentario:)

Bases de Datos XML

Consultas complejas:

(A) Join de documentos:

Ejemplo de la relación, por Deptno en nuestro ejemplo de empleados:

```
for $emp in //Empleados/empleado
```

```
let $emple:=$emp/nombre
```

```
let $dep:=$emp/dep
```

```
let $dnom:=(//Departamentos/departamento[DEPTNO=$dep]/NOMBRE)
```

```
return <res>{$emple,$dep,$dnom}</res>
```

Bases de Datos XML

The screenshot displays the eXide web application interface. The browser address bar shows the URL `http://localhost:9090/exist/apps/eXide/index.html`. The application has a menu bar with options: File, Edit, Navigate, Buffers, Application, XQuery, Help, and a status bar indicating "Logged in as admin". Below the menu is a toolbar with buttons: New, New XQuery, Open, Save, Close, Eval, and Run. The left sidebar shows a file tree under the "db" directory, including "apps", "Pruebas", "Departamentos.xrn", "Empleados.xml", "consulta1", "consulta2", "consulta3", "demo_customers.x", "XSLTForms-Demo", "betterform", "dashboard", "demo", "doc", "eXide", "fundocs", "markdown", "monex", "shared-resources", "xsltforms", and "system". The main editor area shows an XQuery in the "consulta3*" tab:

```
1 for $emp in //Empleados/empleado
2 let $emple:=$emp/nombre
3 let $dep:=$emp/dep
4 let $dnom:=(//Departamentos/departamento[DEPTNO=$dep]/NOMBRE)
5 return <res>{$emple,$dep,$dnom}</res>
```

Below the editor, the output of the query is displayed in the "XML Output" tab for the file `/db/apps/Pruebas/consulta3`. The output shows three XML records:

```
<res>
  <nombre>David</nombre>
  <dep>10</dep>
  <NOMBRE>Marketing</NOMBRE>
</res>
<res>
  <nombre>Eduardo</nombre>
  <dep>2</dep>
</res>
<res>
  <nombre>Joseph</nombre>
  <dep>3</dep>
```

Bases de Datos XML

(B) Sentencias de actualización de eXist.

Insert

- Update insert ELEMENTO into EXPRESION
- Update insert ELEMENTO following EXPRESION
- Update insert ELEMENTO preceding EXPRESION

Replace

- Update replace NODO with VALOR_NUEVO

Value

- Update VALUE NODO with "VALOR_NUEVO"

Delete

- Update delete expresion

Rename

- Update rename NODO as Nuevo_NOMBRE

Bases de Datos XML

Ejemplos:

- ❖ **Inserta** un nuevo departamento en la última posición =>

```
update insert <departamentos><depno>99</depno>  
<nombre>Danza</nombre>  
<localidad>Tejeda</localidad></departamentos>  
into //Departamentos
```

- ❖ **Modifica** el salario de los empleados del departamento 10, incrementándoles 500€.

```
for $emp in //Empleados/empleado[dep=10]  
let $sal:=$emp/salario  
return  
update value $emp/salario with data($sal)+500
```

- ❖ **Elimina** al empleado con id=80

```
update delete //Empleados/empleado[id=80]
```